

Dynamic Sparsity in NLP

Edoardo M. Ponti

ATNLP Guest Lecture, 30 January 2025

Based on a NeurIPS 2024 Tutorial



Presenters: André F. T. Martins and Edoardo M. Ponti.

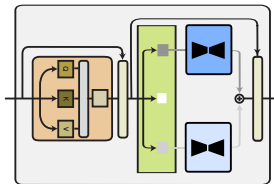
<https://dynamic-sparsity.github.io/>

Outline

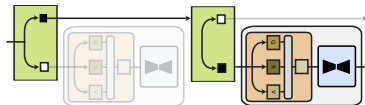
1 Sparse Architectures of Foundation Models

- Mixtures of Experts
- Sparse Layers and Tokens
- Sparse Attention

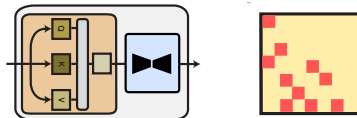
Experts



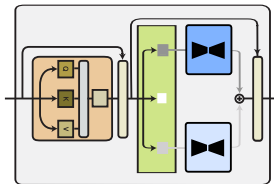
Depth



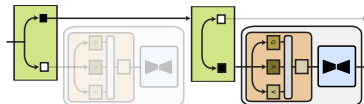
Attention



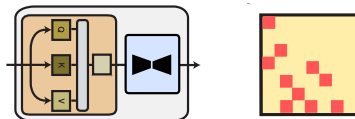
Experts



Depth



Attention



Outline

1 Sparse Architectures of Foundation Models

- Mixtures of Experts
- Sparse Layers and Tokens
- Sparse Attention

Why Sparse MoEs?

- MoEs in a nutshell: input-dependent weighted average of the output of multiple models (aka experts)
- For the original MoEs, [Jacobs et al. \(1991a\)](#) listed these advantages:
 - learning speed,
 - better generalisation,
 - more interpretable representations,
 - reduced hardware requirements.
- Core idea for sparse MoEs ([Shazeer et al., 2017](#)):
 - increase the total parameters of foundation models while retaining efficiency → better **scaling laws** ([Kaplan et al., 2020](#))
 - Still, high memory footprint!

Mixtures of Experts (MoEs) (Jacobs et al., 1991b)

Given input $\mathbf{x}$ and **experts** $\mathcal{E} = \{f_1, \dots, f_n\}$, a **gating function** (or **router**) returns the ‘importance’ of each expert:

$$p(\mathcal{E} \mid \mathbf{x}) = g(\mathbf{x}) = \text{softmax}[\text{MLP}(\mathbf{x})]$$

The output is the linear combination of the outputs of each expert:

$$\hat{y} = \sum_{i=1}^{|\mathcal{E}|} g(\mathbf{x})_i f_i(\mathbf{x})$$

MoEs (continued)

This encourages **competition** between experts, leading to specialisation.

Why? Let's take a look at the derivative of an MSE loss ℓ wrt $g(\mathbf{x})_i$:

$$\frac{\partial \ell}{\partial g(\mathbf{x})_i} = (\hat{y} - y)g(\mathbf{x})_i \left(f_i(\mathbf{x}) - \sum_j f_j(\mathbf{x})g(\mathbf{x})_j \right)$$

The sign also depends on the margin between the output expert i and the others combined.

Another perspective: implicitly **partitioning the data**

In the limit where $p(\mathcal{E} \mid \mathbf{x})$ is deterministic, each $\mathcal{E}_i$ is trained on the dataset $\mathcal{D} \triangleq \{\mathbf{x} \mid g(\mathbf{x})_i = 1\}$. Probabilistic routing defines a soft partition.

Deep MoEs (Eigen et al., 2013)

Next innovation: instead of experts being whole models, stack multiple layers of MoEs

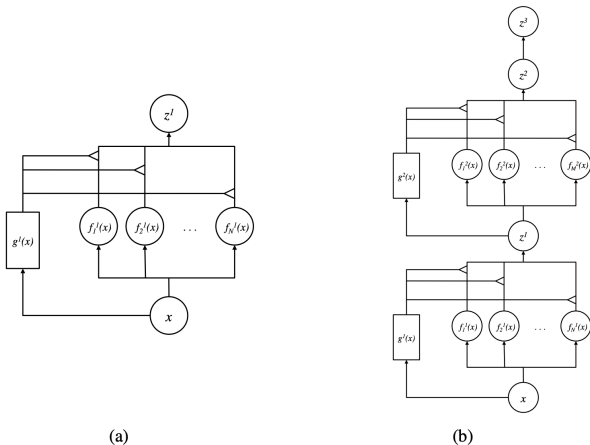
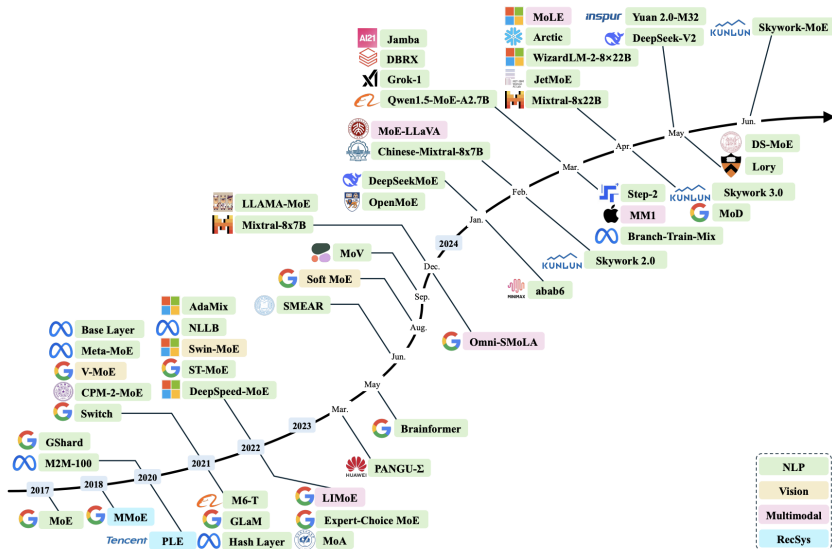


Figure 1: (a) Mixture of Experts; (b) Deep Mixture of Experts with two layers.

History of Sparse MoEs (Cai et al., 2024)



Several key innovations:

- Introduce MoE layers in **LSTM** LMs, with **token-level routing**
- **Top- k routing** for **conditional computation** (ϵ is random noise):

$$p(\mathcal{E} \mid \mathbf{x}) = \text{softmax}(\mathbf{z}) \quad z_i = \begin{cases} g(\mathbf{x})_i & \text{if } i \in \arg \text{top-}k[g(\mathbf{x}) + \epsilon] \\ -\infty & \text{else} \end{cases}$$

- **Load balancing** loss to optimise expert usage during training

Challenges of Sparse MoE LM

- 1
 - **Shrinking batch problem**: each expert observes only $\frac{kb}{n}$ tokens among the b tokens in a batch by selecting k out of n experts.
 - Solution: expand the batch size with **mixed parallelism**: d devices serve
 - as *data-parallel* replicas for standard / routing layers;
 - as *expert-parallel* shards for experts (note: this is different from *model parallelism* of FFNs!)

→ each expert observes d more tokens
- 2
 - **Module collapse**: rich-gets-richer effect for few experts.
Underutilisation of resources due to uneven routing.
 - Solution: **auxiliary losses** based on **importance** and **load**

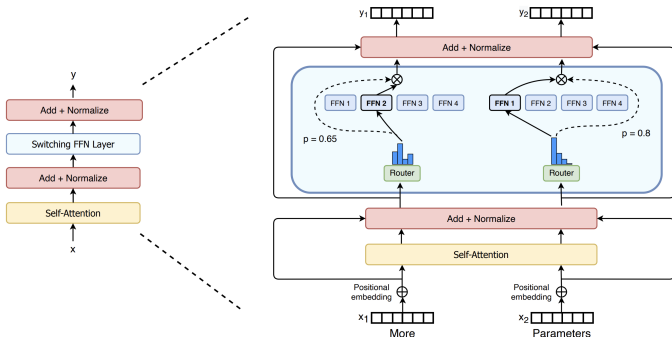
Switch Transformer (Fedus et al., 2022)



Building on G-Shard (Lepikhin et al., 2021), experts are now **FFNs in Transformers** and routing is **top-1**.

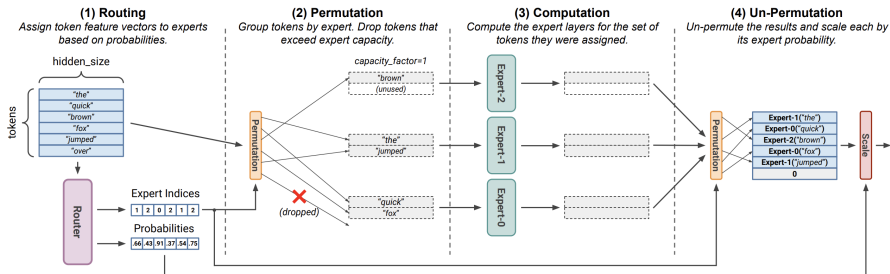
Tensor sizes are static but routing choices are dynamic → trade-off expert underusing (padding) and overusing (token dropping) via **expert capacity**.

$$\text{expert capacity} = \frac{\text{tokens per batch}}{\text{number of experts}} \times \text{capacity factor}$$



Jupyter notebook
on our tutorial
webpage!

Sparse MoE Workflow



How to ensure that computation is not wasted and information is not discarded?

Two directions:

- 1 balancing losses
- 2 block-sparse matrix multiplication (Gale et al., 2023)

Balancing Losses

Aux loss (Lepikhin et al., 2021; Fedus et al., 2022):

$$\ell_A = \frac{1}{|\mathcal{E}|} \sum_{j=1}^{|\mathcal{E}|} \frac{c_j}{b} \left(\frac{1}{b} \sum_{i=1}^b g(x_i)_j \right)$$

where c_j is the number of tokens assigned to expert $\mathcal{E}_j$ and $g(x_i)$ the router scores. Simplifies Shazeer et al. (2017)'s losses.

Router z loss (Zoph et al., 2022):

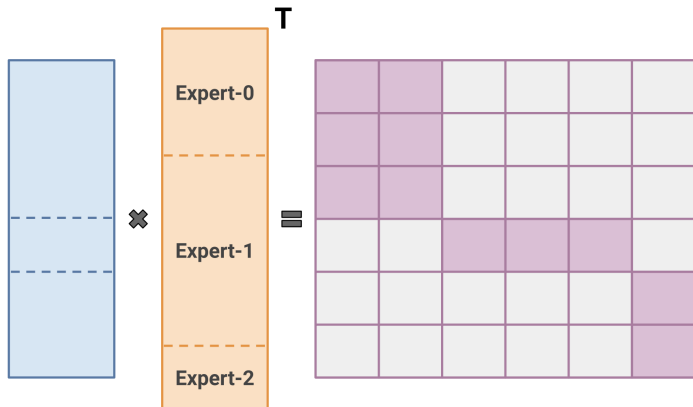
$$\ell_Z = \frac{1}{b} \sum_{i=1}^b \left(\log \sum_{j=1}^{|\mathcal{E}|} \exp[g(x_i)_j] \right)^2$$

Intuition: avoid extreme router logits for increased stability

Overcoming the Hardware Lottery

Hardware lottery (Hooker, 2021): model design is due to its compatibility with existing hardware, rather than its inherent superiority.

Megablocks (Gale et al., 2023): block sparse matrix multiplication makes load-balancing unnecessary → MoE is more expressive



Will Sparse MoE become the 'default' LLM architecture?



Bearish

Bullish

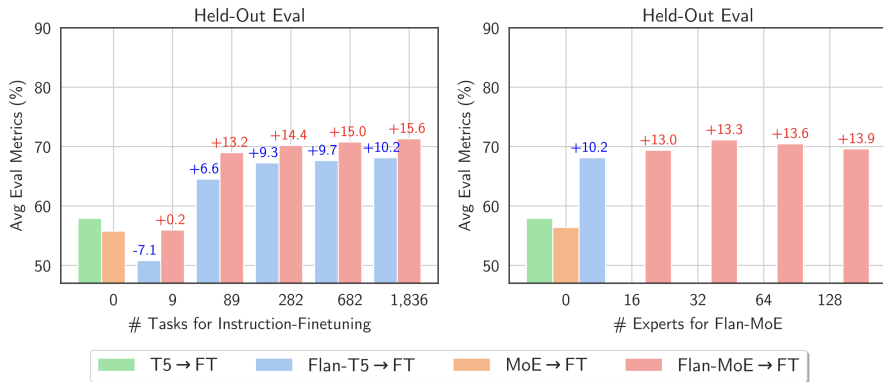
- Instruction tuning better than dense (Shen et al., 2024)
- Scaling laws better than dense (Ludziejewski et al., 2024)

- Cost–Accuracy–Performance trade-off (Fu et al., 2024)
- Limited expert specialisation (Muennighoff et al., 2024; Jiang et al., 2024a)
- Discrepancy in factual vs reasoning tasks (Jelassi et al., 2024)

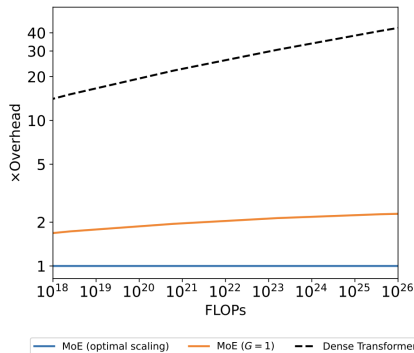
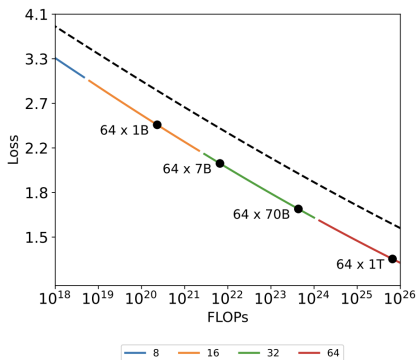
Instruction Tuning Sparse MoEs (Shen et al., 2024)

Sparse MoEs were believed to be harder to fine-tune (Artetxe et al., 2022)

However, *instruction* tuning is more beneficial than in dense models!

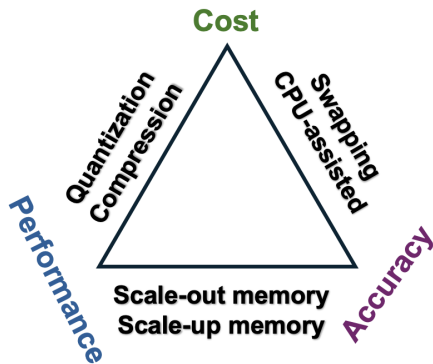


Scaling Laws for MoEs (Ludziejewski et al., 2024)



- Sparse MoEs offer a better performance–efficiency Pareto front compared to dense models
- The gap widens as we scale model size and data
- Optimal *granularity* G (ratio of d_{FFN} to d_{expert}) varies

MoE-CAP: A Benchmark for Cost, Accuracy, and Performance in Sparse MoEs (Fu et al., 2024)



- New **sparsity-aware** metrics for **performance** (=efficiency):
Memory Bandwidth Utilization and Model FLOPS Utilization
- New **cost** metric aware of **heterogeneous computing resources**, e.g., off-load experts from HBM to DRAM and SSD and/or compute to CPU.
- **Accuracy** benchmarking on MMLU, GSM8K, Arena-Hard.

<https://github.com/Auto-CAP/MoE-CAP>

CAP Trade-offs

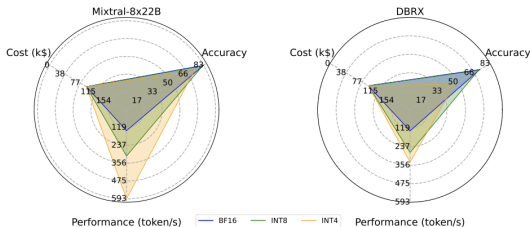


Figure 4: Comparison of CA and CP systems on GSM8K dataset under CAP view.

Quantisation

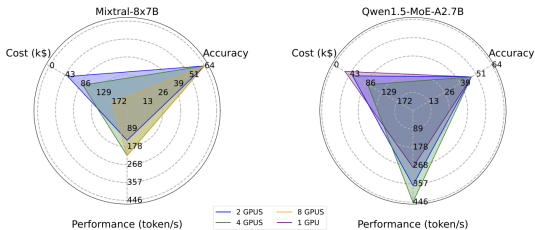
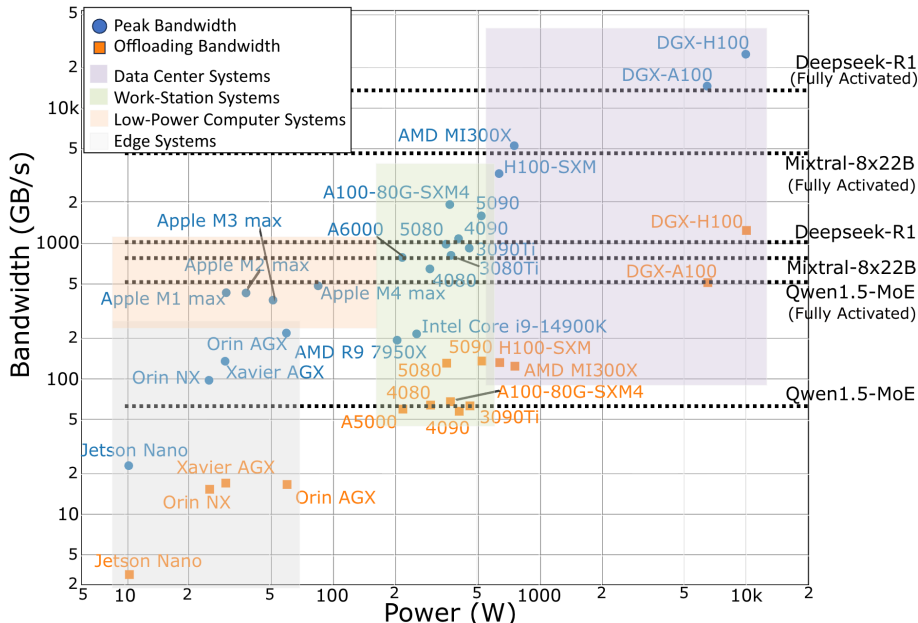


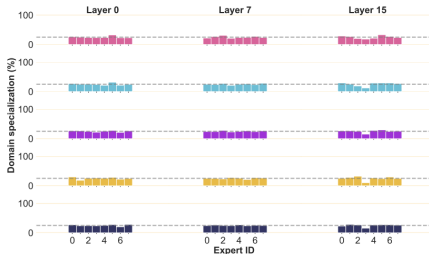
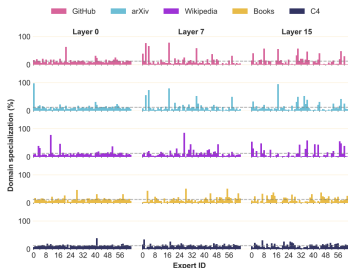
Figure 5: Comparison of PA and AC systems on GSM8K under CAP method.

Parallelisation

Deploying Sparse MoEs on Heterogeneous Hardware



Do Experts Specialise? OLMoE has some answers.



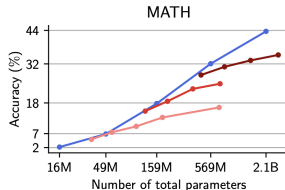
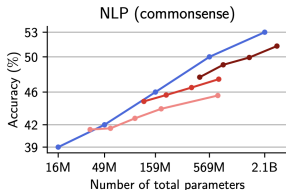
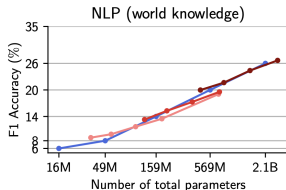
Comparing OLMoE (**left**) (Muennighoff et al., 2024) and Mixtral (**right**) (Jiang et al., 2024a), it looks like specialisation may depend on the **expert inventory size** or on **upcycling** (here, from dense Mistral)

Also: few strong co-activation among experts in one layer (no redundancy?)

Vocabulary specialization in later layers (but based on *output* token id)

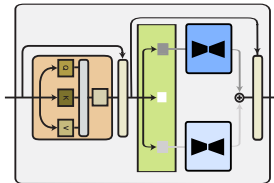
Mixture of parrots (Jelassi et al., 2024)

Sparse MoEs benefit **factual** tasks (by having larger parameter capacity to memorise information), but not necessarily **reasoning** tasks

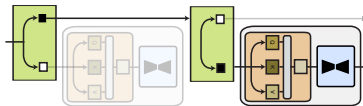


■ Dense transformer ■ MoE (18M active parameters) ■ MoE (58M active parameters) ■ MoE (200M active parameters)

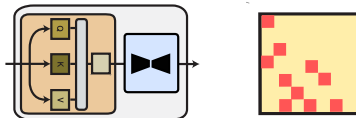
Experts ✓



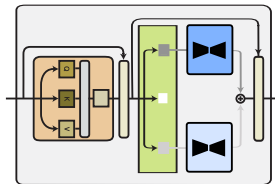
Depth



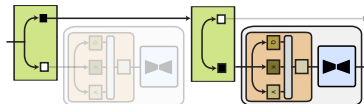
Attention



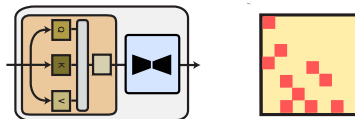
Experts ✓



Depth



Attention



Outline

1 Sparse Architectures of Foundation Models

- Mixtures of Experts
- Sparse Layers and Tokens
- Sparse Attention

Conditional Computation (Bengio et al., 2013)

Conditional computation: “gaters that can turn off in combinatorially many ways large chunks of the computation performed in the rest of the neural network”. Two slightly different interpretations:

- 1 Find a better accuracy-efficiency trade-off with a fixed sparsity level;
- 2 Adjust the computation graph according to the problem complexity.

Gradients for sparsification / conditional computation decisions are non-informative $\rightarrow$ exploration of **gradient estimators** (Bengio et al., 2013):

For $y = \sigma(x)$, $\hat{y} = \text{round}(y)$, $\mathcal{L} = f(\hat{y})$:

- REINFORCE: $\nabla_x \mathcal{L} \approx \mathcal{L} \cdot (\hat{y} - \sigma(x))$
- **straight-through estimator:** $\frac{\delta y}{\delta \hat{y}} \approx 1$, so $\nabla_x \mathcal{L} \approx \frac{\delta \mathcal{L}}{\delta \hat{y}} \frac{\delta y}{\delta x}$
- ...see *Stochastic* (Krieken et al., 2021) for many more!

Routes to Conditional Computation

The idea is pre-neural! (Viola and Jones, 2001)

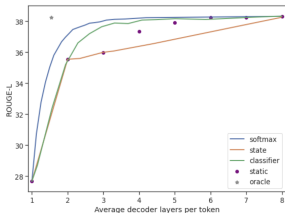
At **inference time**:

- 1 **Early exit**: Make predictions before reaching the final layer
 - Possibly different depths for different image patches / text tokens / etc.
- 2 **Layer skipping**: For every layer, decide whether to use it or skip it
- 3 **Token merging**: merge groups of tokens to reduce the sequence length.
 - Group length can be fixed or dynamic.

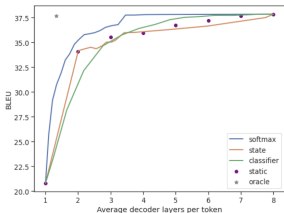
Confident Adaptive Language Models (Schuster et al., 2022)

Comparison of early exit strategies:

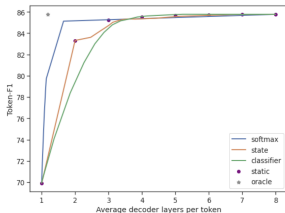
- 1 Entropy: extra FLOPS but parallelisable
- 2 Saturation in hidden states of consecutive layers: fast and parameter-free
- 3 Parametric halting classifier, trained post-hoc via consistency oracle



(a) CNN/DM



(b) WMT



(c) SQUAD

Principled strategy with guarantees (CALM)

Early-exit models should:

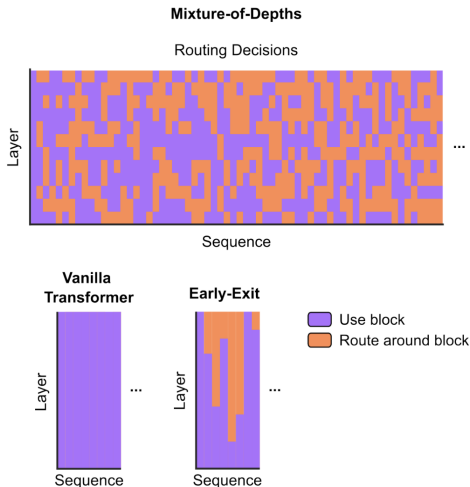
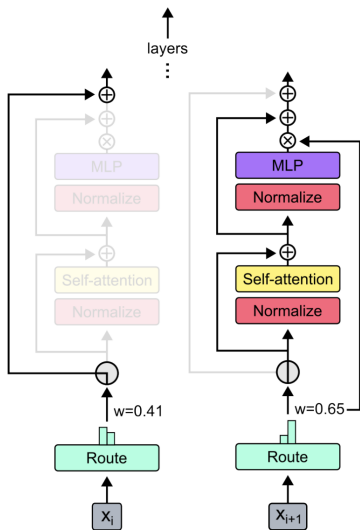
$p(\mathbb{E}[d(\hat{\mathbf{y}}_{\text{early}}, \hat{\mathbf{y}}_{\text{full}})] \leq \delta \mid \mathbf{x}) \geq 1 - \epsilon$ (match outputs of full models)

$p(\mathbb{E}[\mathcal{L}(\hat{\mathbf{y}}_{\text{early}}, \mathbf{y}) - \mathcal{L}(\hat{\mathbf{y}}_{\text{full}}, \mathbf{y})] \leq \delta \mid \mathbf{x}) \geq 1 - \epsilon$ (match loss of full models)

How to pick threshold τ that satisfies δ and ϵ ? Hard because non-monotonic.

Learn Then Test (LTT) framework: cast hyper-parameter selection as a multiple testing problem based on a calibration set of inputs-outputs

Mixture of Depths (MoDs) (Raposo et al., 2024)

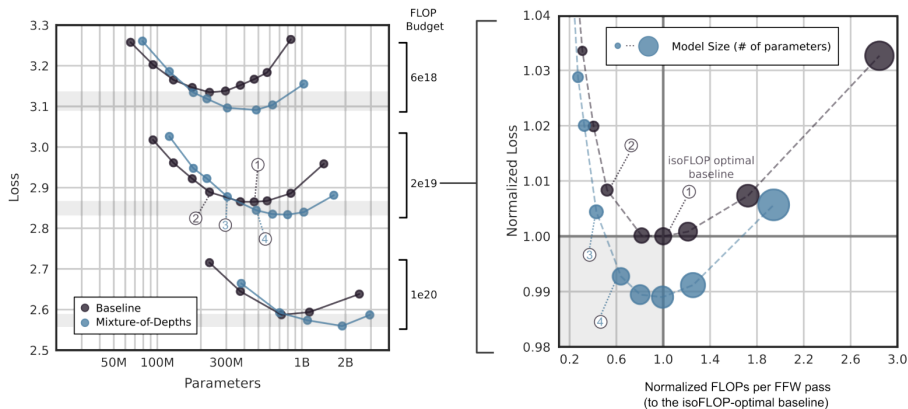


Contrary to early exit, in MoDs routers determine layer skipping.

Mixture of Depths (MoDs)

- Not just for MLPs but also self-attention (contrary to MoEs)
- MoD caps the number of tokens per layer via **top- k learned routing**.
- How to do this during generation? Top- k is **non-autoregressive**. Two solutions: auxiliary autoregressive router or auxiliary loss to 'quantise' routing scores
- The secret sauce is the **static** computation graph with a **dynamic** choice of tokens → the hardware lottery strikes again!

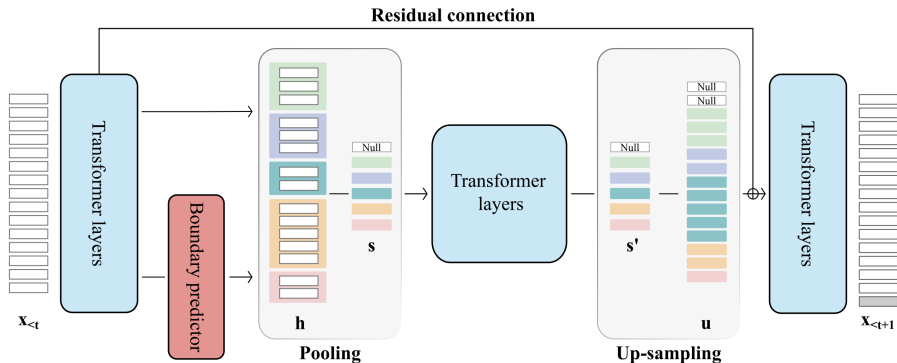
IsoFLOP Analysis



Some MoD variants are both faster to step and better performing than the isoFLOP optimal baseline.

Don't skip, merge! (Nawrot et al., 2023)

Merge contiguous tokens: **dynamic token pooling** reduces attention complexity by (compression ratio)², extending fixed pooling in Hourglass Transformers (Nawrot et al., 2022).



Efficient during both training and inference!

Autoregressivity is preserved with special tokens ([Null] in the image).

Boundary Predictor for Merging

The **boundary predictor** can be learned:

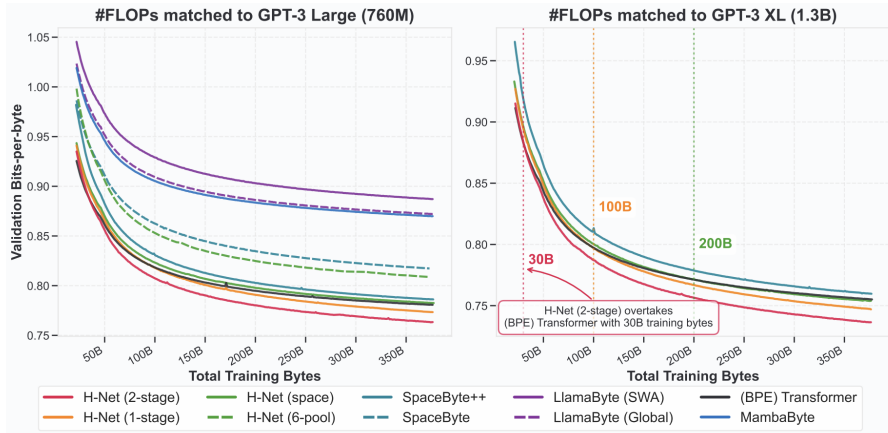
- 1 end-to-end via stochastic reparameterisation;
- 2 relying on explicit data boundaries (e.g., whitespaces);
- 3 via supervision from Unigram tokenizer;
- 4 via supervision from entropy spikes.

An auxiliary loss controls the **shortening factor** (or **compression ratio**).

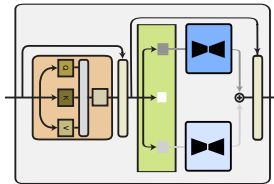
The best of both worlds: better performance *and* efficiency!

Enhancements and Scaling of Token Merging

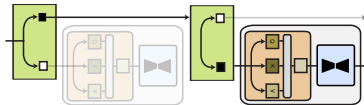
Recently, Byte Latent Transformer (BLT) (Pagnoni et al., 2025) and H-Net (Anonymous, 2026) scaled up this idea beyond 1B scale.



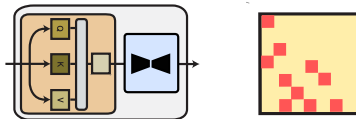
Experts ✓



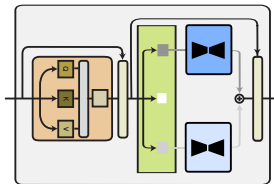
Depth ✓



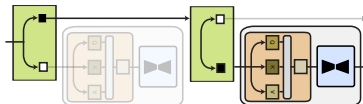
Attention



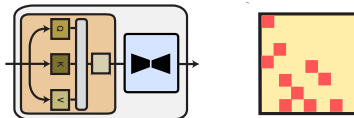
Experts ✓



Depth ✓



Attention



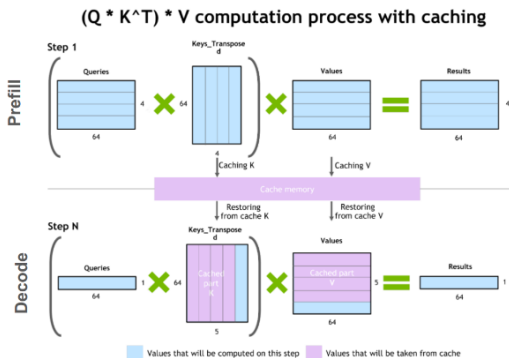
Outline

1 Sparse Architectures of Foundation Models

- Mixtures of Experts
- Sparse Layers and Tokens
- Sparse Attention

Two main phases for LLM deployment

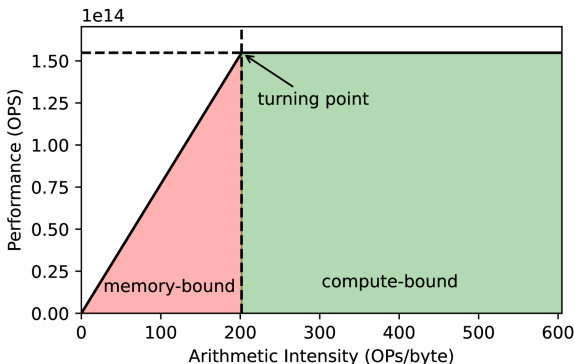
- **Prefilling:** compute keys and values for the input
- **Generation:** autoregressive output generation



From <https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/>

Challenges of Each Phase

- **Prefilling:** computation heavy, dominated by the quadratic complexity of attention wrt the sequence length
- **Generation:** linear wrt sequence length but the key-value cache grows linearly. Attention is **memory-bound** (retrieving keys and values from memory takes more time than the actual computation)



We can sparsify keys-values to alleviate these issues:

- **Prefilling:** identify a subset of important token interactions and approximate the attention output with this subset.
- **Generation:** reduce the memory footprint by discarding less important tokens

(Also see ∞ -former and sparse attention for interpretability in **Part I!**)

Other benefits include extending LLM “horizon”:

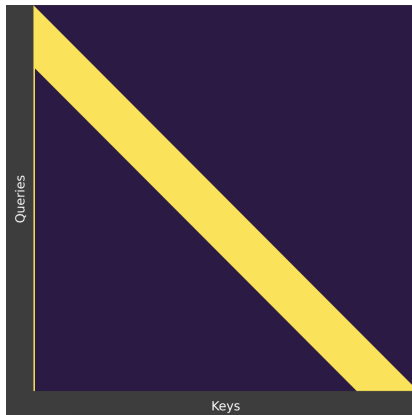
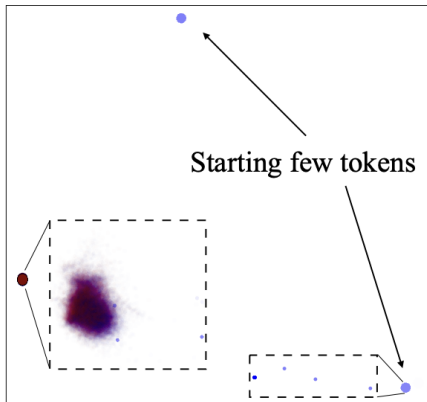
- Prefilling for **long context**
- Generation for **chain-of-thought reasoning**



Check out the Jupyter notebook on our tutorial webpage!

Static sparsity in Prefilling

- **Local Window**: This reflects a Markov assumption where most recent keys-values are always retained
- **Sink Tokens** (Xiao et al., 2024): initial tokens have strong attention scores (independent of semantics) and are differently distributed (Han et al., 2024) (left plot)



Vertical+Slash Attention (MInference) (Jiang et al., 2024b)

- 1 calculates partial attention scores
- 2 Then creates a mask combining two types of connectivity:
 - Attention to top-K most relevant tokens (content-based)
 - Attention along top-K diagonal stripes (structure-based)

Algorithm 2 Vertical-Slash Head

Input: $Q, K, V \in \mathbb{R}^{S \times d_h}$, $k_v, k_s \in \mathbb{N}$

Approximate vertical and slash pattern (last_q = 64)

$$\hat{A} \leftarrow \text{softmax} \left(Q_{[-\text{last_q}:]} K^\top / \sqrt{d} + m_{\text{casual}} \right)$$

Indices of top k_v vertical line, sum in vertical

$$i_v \leftarrow \text{argtopk} \left(\text{sum}_v(\hat{A}), k_v \right)$$

Indices of top k_s slash line, sum in slash

$$i_s \leftarrow \text{argtopk} \left(\text{sum}_s(\hat{A}), k_s \right)$$

Build sparse attention index

$$i_{vs} \leftarrow \text{sparseformat}(i_v, i_s)$$

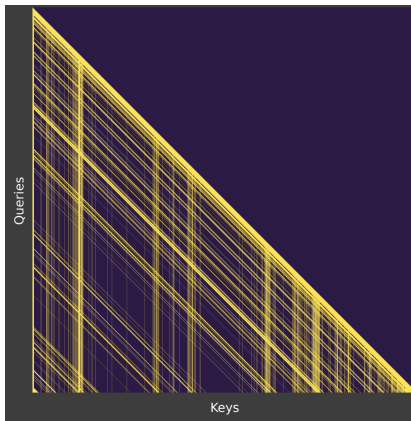
Final dynamic sparse attention scores (only index block)

$$A \leftarrow \text{softmax} \left(\text{sparse}(QK^\top, i_{vs}) / \sqrt{d} \right)$$

Sparse mixed scores and values

$$y \leftarrow \text{sparse}(AV, i_{vs})$$

return y



Block Sparse Attention (Jiang et al., 2024b)

- 1 Divides the sequence into fixed-size chunks.
- 2 Computes chunk-level attention scores with averaged token representations.
- 3 Allows each chunk to attend to the top-K most relevant chunks.

Algorithm 3 Block-Sparse Head

Input: $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{S \times d_h}$, $k_b \in \mathbb{N}$

Approximate block-sparse pattern (block_size = 64)

$\hat{\mathbf{Q}} \leftarrow \text{MeanPooling}(\mathbf{Q}, \text{block_size})$

$\hat{\mathbf{K}} \leftarrow \text{MeanPooling}(\mathbf{K}, \text{block_size})$

$\hat{\mathbf{A}} \leftarrow \text{softmax} \left(\hat{\mathbf{Q}} \hat{\mathbf{K}}^\top / \sqrt{d} + \mathbf{m}_{\text{casual}} \right)$

Indices of top k blocks

$\mathbf{i}_b \leftarrow \text{argtopk} \left(\hat{\mathbf{A}}, k_b \right)$

Build sparse attention index

$\mathbf{i}_b \leftarrow \text{sparseformat}(\mathbf{i}_b)$

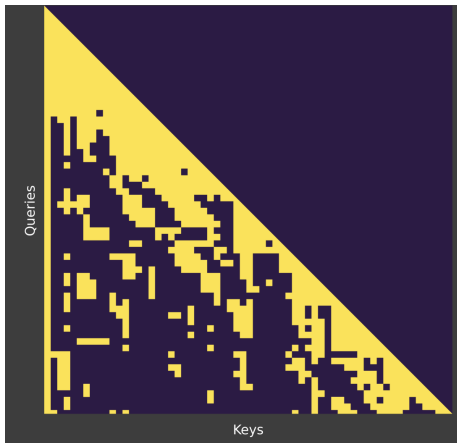
Final dynamic sparse attention scores (only index block)

$\mathbf{A} \leftarrow \text{softmax} \left(\text{sparse}(\mathbf{Q} \mathbf{K}^\top, \mathbf{i}_b) / \sqrt{d} \right)$

Sparse mixed scores and values

$\mathbf{y} \leftarrow \text{sparse}(\mathbf{A} \mathbf{V}, \mathbf{i}_b)$

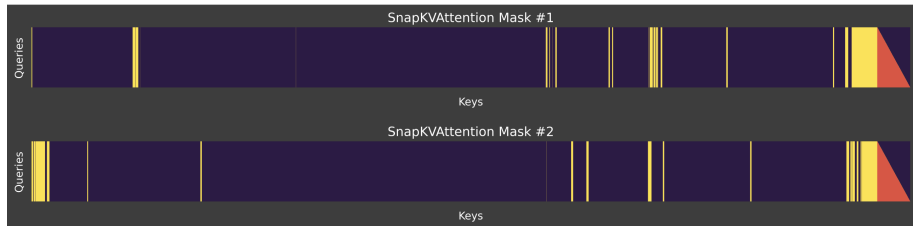
return $\mathbf{y}$



SnapKV Attention (Li et al., 2024)

Now turning to sparse attention during **generation**!

- 1 Approximates attention scores using a suffix window of queries
- 2 Uses attention score pooling to identify important token clusters
- 3 Keeps only the most relevant tokens plus a recent window

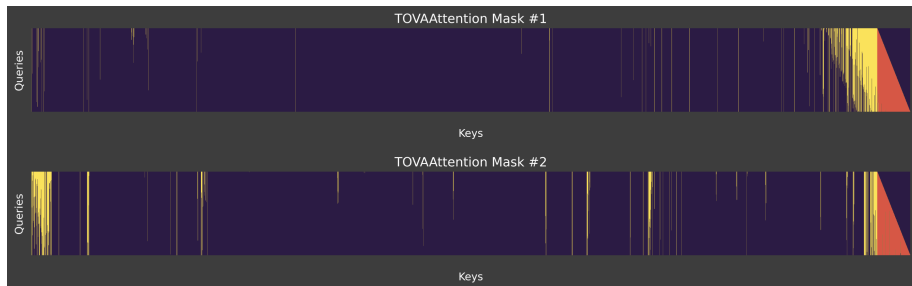


Interaction colours: **Red** (allowed generation query – generation key),
Yellow (allowed generation query – prefilling key), **Blue** (forbidden)

TOVA Attention (Oren et al., 2024)

Dynamically prunes the KV cache during generation:

- 1 Uses the last prefilling token to identify initially important tokens
- 2 During generation, maintains a fixed-size cache by removing the least attended token after processing each new token

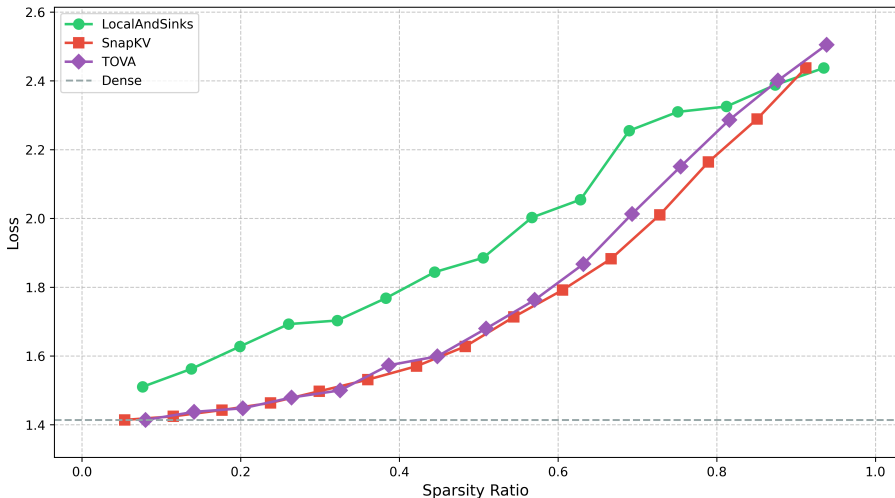


Note some vertical lines are cut off!

Comparison on Summarisation Task

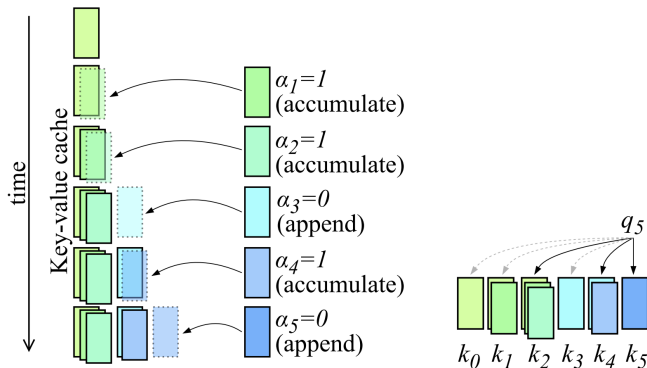
- Dynamic methods define a better Pareto frontier than static ones
- Yet, they still degrade performance beyond low sparsity ratios

Comparison of Sparse Attention Methods



Dynamic Memory Compression (Nawrot et al., 2024)

- Compress representations instead of dropping them.
- Decide if to **append** or **accumulate** new keys-values to the cache



Generation (left) vs Training (right)

During training, the binary decision variable α is **continuously relaxed**

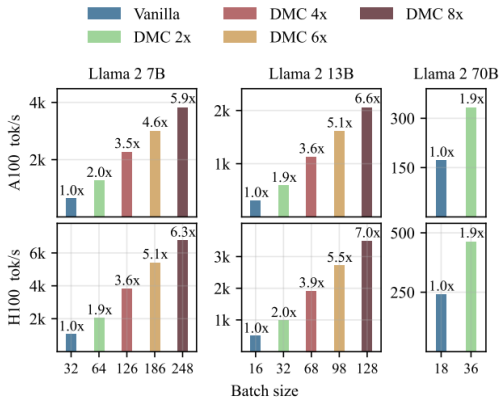
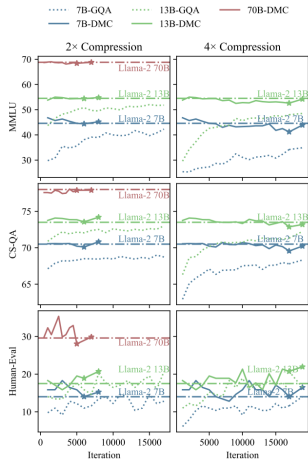
To parallelise training, partly accumulated tokens are kept, too

α modulates query-key interactions through an **extra mask** to progressively remove items inaccessible at inference time

$$\begin{aligned}
 z_0 &\leftarrow \omega_0, & z_i &\leftarrow z_{i-1}\alpha_i + \omega_i, \\
 \mathbf{k}_0 &\leftarrow \mathbf{k}_0, & \mathbf{k}_i &\leftarrow \frac{\alpha_i \mathbf{k}_{i-1} z_{i-1} + \mathbf{k}_i \omega_i}{z_i}, \\
 \mathbf{v}_0 &\leftarrow \mathbf{v}_0, & \mathbf{v}_i &\leftarrow \frac{\alpha_i \mathbf{v}_{i-1} z_{i-1} + \mathbf{v}_i \omega_i}{z_i}.
 \end{aligned}
 \quad
 \begin{matrix}
 & \mathbf{k}_0 & \mathbf{k}_1 & \mathbf{k}_2 & \dots & \mathbf{k}_n \\
 \mathbf{q}_0 & \begin{bmatrix} 0 & -\infty & -\infty & \dots & -\infty \end{bmatrix} \\
 \mathbf{q}_1 & \begin{bmatrix} \log(1 - \alpha_1) & 0 & -\infty & \dots & -\infty \end{bmatrix} \\
 \mathbf{q}_2 & \begin{bmatrix} \log(1 - \alpha_1) & \log(1 - \alpha_2) & 0 & \dots & -\infty \end{bmatrix} \\
 \vdots & \begin{bmatrix} \vdots & & & \ddots & \vdots \end{bmatrix} \\
 \mathbf{q}_n & \begin{bmatrix} \log(1 - \alpha_1) & \log(1 - \alpha_2) & \log(1 - \alpha_3) & \dots & 0 \end{bmatrix}
 \end{matrix}$$

Foundation models can be **retrofitted** with DMC behaviour with negligible extra data

Results on LLM Benchmarks



Resources

- Training-free methods for memory sparsification:
<https://github.com/PiotrNawrot/sparse-frontier>
- Retrofitting for memory compression:
<https://github.com/NVIDIA/Megatron-LM/tree/dmc>

Summary

- The *why* and *where* of sparsity are connected: in **experts** and **depth** for scaling and conditional computation, in **adapters** for **generalisation**, in **attention** for throughput/latency.
- Will Sparse MoEs become the default? Scaling laws and post-training are reasons for optimism, but CAP trade-offs, lack of specialisation, and factual vs reasoning tasks should make us cautious.
- Mixtures-of-adapters could unlock the potential of asynchronous and independently developed, massively multitask LLMs.
- Sparse memory has different challenges and solutions in **prefilling** and **generation** stages → may be the key to long context and long reasoning.
- Dynamic **sparsification** is a special case of dynamic **compression**, which requires extra training but doesn't degrade performance.
- The hardware lottery still plays a significant role in the design of MoEs and attention.

References I

- Anonymous (2026). Dynamic chunking for end-to-end hierarchical sequence modeling. In *The Fourteenth International Conference on Learning Representations*.
- Artetxe, M., Bhosale, S., Goyal, N., Mihaylov, T., Ott, M., Shleifer, S., Lin, X. V., Du, J., Iyer, S., Pasunuru, R., Anantharaman, G., Li, X., Chen, S., Akin, H., Baines, M., Martin, L., Zhou, X., Koura, P. S., O'Horo, B., Wang, J., Zettlemoyer, L., Diab, M., Kozareva, Z., and Stoyanov, V. (2022). Efficient large scale language modeling with mixtures of experts. In Goldberg, Y., Kozareva, Z., and Zhang, Y., editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 11699–11732, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.
- Bengio, Y., Léonard, N., and Courville, A. (2013). Estimating or propagating gradients through stochastic neurons for conditional computation. *preprint arXiv:1308.3432*.
- Cai, W., Jiang, J., Wang, F., Tang, J., Kim, S., and Huang, J. (2024). A survey on mixture of experts. *arXiv preprint arXiv:2407.06204*.
- Eigen, D., Ranzato, M., and Sutskever, I. (2013). Learning factored representations in a deep mixture of experts. *arXiv preprint arXiv:1312.4314*.
- Fedus, W., Zoph, B., and Shazeer, N. (2022). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.
- Fu, Y., Jiang, Y., Huang, Y., Nie, P., Lu, Z., Xue, L., He, C., Sit, M.-K., Xue, J., Dong, L., Miao, Z., Zou, K., Ponti, E., and Mai, L. (2024). MoE-CAP: Cost-accuracy-performance benchmarking for mixture-of-experts systems.

References II

- Gale, T., Narayanan, D., Young, C., and Zaharia, M. (2023). Megablocks: Efficient sparse training with mixture-of-experts. In Song, D., Carbin, M., and Chen, T., editors, *Proceedings of Machine Learning and Systems*, volume 5, pages 288–304. Curan.
- Han, C., Wang, Q., Peng, H., Xiong, W., Chen, Y., Ji, H., and Wang, S. (2024). LM-infinite: Zero-shot extreme length generalization for large language models. In Duh, K., Gomez, H., and Bethard, S., editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3991–4008, Mexico City, Mexico. Association for Computational Linguistics.
- Hooker, S. (2021). The hardware lottery. *Commun. ACM*, 64(12):58–65.
- Jacobs, R. A., Jordan, M. I., and Barto, A. G. (1991a). Task decomposition through competition in a modular connectionist architecture: The what and where vision tasks. *Cognitive science*, 15(2):219–250.
- Jacobs, R. A., Jordan, M. I., Nowlan, S. J., and Hinton, G. E. (1991b). Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87.
- Jelassi, S., Mohri, C., Brandfonbrener, D., Gu, A., Vyas, N., Anand, N., Alvarez-Melis, D., Li, Y., Kakade, S. M., and Malach, E. (2024). Mixture of parrots: Experts improve memorization more than reasoning. *arXiv preprint arXiv:2410.19034*.
- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., Casas, D. d. I., Hanna, E. B., Bressand, F., et al. (2024a). Mixtral of experts. *arXiv preprint arXiv:2401.04088*.
- Jiang, H., Li, Y., Zhang, C., Wu, Q., Luo, X., Ahn, S., Han, Z., Abdi, A. H., Li, D., Lin, C.-Y., Yang, Y., and Qiu, L. (2024b). MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

References III

- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models.
- Krieken, E., Tomczak, J., and Ten Teije, A. (2021). Stochastic: A framework for general stochastic automatic differentiation. *Advances in Neural Information Processing Systems*, 34:7574–7587.
- Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. (2021). GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*.
- Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. (2024). SnapKV: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*.
- Ludziejewski, J., Krajewski, J., Adamczewski, K., Pióro, M., Krutul, M., Antoniak, S., Ciebiera, K., Król, K., Odrzygóźdź, T., Sankowski, P., Cygan, M., and Jaszczur, S. (2024). Scaling laws for fine-grained mixture of experts. In *Forty-first International Conference on Machine Learning*.
- Muennighoff, N., Soldaini, L., Groeneveld, D., Lo, K., Morrison, J., Min, S., Shi, W., Walsh, P., Tafjord, O., Lambert, N., et al. (2024). OLMoE: Open mixture-of-experts language models. *arXiv preprint arXiv:2409.02060*.
- Nawrot, P., Chorowski, J., Lancucki, A., and Ponti, E. M. (2023). Efficient transformers with dynamic token pooling. In Rogers, A., Boyd-Graber, J., and Okazaki, N., editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6403–6417, Toronto, Canada. Association for Computational Linguistics.
- Nawrot, P., Łańcucki, A., Chochowski, M., Tarjan, D., and Ponti, E. (2024). Dynamic memory compression: Retrofitting LLMs for accelerated inference. In *Forty-first International Conference on Machine Learning*.

References IV

- Nawrot, P., Tworkowski, S., Tyrolski, M., Kaiser, Ł., Wu, Y., Szegedy, C., and Michalewski, H. (2022). Hierarchical transformers are more efficient language models. In *Findings of the Association for Computational Linguistics: NAACL 2022*, pages 1559–1571.
- Oren, M., Hassid, M., Yarden, N., Adi, Y., and Schwartz, R. (2024). Transformers are multi-state RNNs. *arXiv preprint arXiv:2401.06104*.
- Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., Zhou, C., Yu, L., Weston, J. E., Zettlemoyer, L., et al. (2025). Byte latent transformer: Patches scale better than tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9238–9258.
- Raposo, D., Ritter, S., Richards, B., Lillicrap, T., Humphreys, P. C., and Santoro, A. (2024). Mixture-of-Depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*.
- Schuster, T., Fisch, A., Gupta, J., Dehghani, M., Bahri, D., Tran, V., Tay, Y., and Metzler, D. (2022). Confident adaptive language modeling. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A., editors, *Advances in Neural Information Processing Systems*, volume 35, pages 17456–17472. Curran Associates, Inc.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*.
- Shen, S., Hou, L., Zhou, Y., Du, N., Longpre, S., Wei, J., Chung, H. W., Zoph, B., Fedus, W., Chen, X., Vu, T., Wu, Y., Chen, W., Webson, A., Li, Y., Zhao, V. Y., Yu, H., Keutzer, K., Darrell, T., and Zhou, D. (2024). Mixture-of-experts meets instruction tuning: A winning combination for large language models. In *The Twelfth International Conference on Learning Representations*.

References V

- Viola, P. and Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. In *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001*, volume 1, pages I–I.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. (2024). Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*.
- Zoph, B., Bello, I., Kumar, S., Du, N., Huang, Y., Dean, J., Shazeer, N., and Fedus, W. (2022). ST-MoE: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*.